

Vareiko Foundation

Comprehensive Usage Guide

Version: 1.0.1

Date: 2026-06-12

Zenject-first architecture for fast, stable Unity project starts.

Foundation Usage Guide (3.0)

This is a practical, end-to-end guide for using `com.vareiko.foundation` as a starter architecture in Unity.

1. What You Get

The package provides:

- 1 deterministic bootstrap and service composition (`FoundationProjectInstaller`, a `VContainerLifetimeScope`);
- 1 scene-local composition (`FoundationSceneInstaller`, a child `LifetimeScope`);
- 1 modular runtime services (save/settings, analytics, backend, monetization, push, attribution, UI, observability);
- 1 typed cloud command service with idempotency/retry queue contracts;
- 1 deterministic RNG streams with snapshot/restore support;
- 1 adapter-oriented integrations (safe null/simulated providers + bridge entry points);
- 1 core gameplay primitives: object pooling, central tick service, `Result<T>`, generic FSM with extensible app states;
- 1 modular assemblies — reference `Vareiko.Foundation.Core` plus the modules you need, or the `Vareiko.Foundation` umbrella for everything;
- 1 built-in runtime tests and validation tooling.

Use it as your baseline layer, then add game-specific domain modules above it.

2. Minimal Integration (Fresh Project)

- 1 Add `FoundationProjectInstaller` to your bootstrap scene (it is the project-scope `LifetimeScope`).
- 2 Add `FoundationSceneInstaller` to gameplay scenes (it parents to the project scope automatically).
- 3 Create and assign these configs first:
- 4 `EnvironmentConfig`
- 5 `ObservabilityConfig`
- 6 `SaveSecurityConfig`
- 7 `AutosaveConfig`
- 8 `BackendCommandConfig` (if you use `ICloudCommandService`)
- 9 `DeterministicRngConfig` (if you use deterministic run seeds)

For the first run keep optional providers in fallback mode:

- 1 backend: None

- 1 IAP: None or Simulated
- 1 ads: None or Simulated
- 1 push: None or Simulated
- 1 attribution: None

3. Installation Dependencies

Required:

- 1 `com.cysharp.unitask`
- 1 `jp.hadashikick.vcontainer`
- 1 `com.cysharp.messagepipe`
- 1 `com.cysharp.messagepipe.vcontainer`
- 1 `com.unity.nuget.newtonsoft-json`
- 1 `com.unity.inputsystem`

Optional production dependencies:

- 1 Unity IAP (UNITY_PURCHASING)
- 1 host push SDK + FOUNDATION_UNITY_NOTIFICATIONS
- 1 PlayFab SDK + PLAYFAB_SDK

4. How Runtime Is Composed

At project startup, `FoundationRuntimeInstaller.InstallProjectServices(...)` installs modules in a strict order (time/common/rng/env/app/bootstrap/config/.../observability).

The order is documented in `Documentation~/ARCHITECTURE.md`.

`FoundationProjectInstaller` exposes serialized config slots and forwards them to the runtime installer.

Since 3.0 the package ships as 10 assemblies (see `ARCHITECTURE.md`, "Assembly Layout"). A subset host references `Vareiko.Foundation.Core` + chosen modules and composes its own project `LifetimeScope`: `RegisterMessagePipe`, the `IFoundationSignalBus` facade, then each module's `RegisterSignals(builder, options) + Install(builder, ...)`. Signal brokers always belong in the project scope.

5. First Play Mode Checklist

- 1 Enter Play Mode in bootstrap scene.
- 2 Verify zero compile errors.
- 3 Verify no startup validation Error.
- 4 Verify boot transitions to expected app state.
- 5 Verify diagnostics snapshot is available via `IDiagnosticsService`.

6. Core Service Usage Patterns

Save/Load

FoundationSaveInstaller binds `ISaveStorage` to `PlayerPrefsSaveStorage` by default. Each slot/key pair is stored as a separate `PlayerPrefs` string containing the current JSON save envelope; rolling backups use separate `.bakN` `PlayerPrefs` keys. Projects that need file-backed saves can pre-bind `ISaveStorage` to `FileSaveStorage` before calling the foundation save installer.

Since 3.0 the default payload serializer is `NewtonsoftJsonSaveSerializer` — dictionaries, nullables and polymorphic payloads serialize correctly. It writes the same `{"Value": ...}` envelope as the previous `JsonUtility` serializer, so older saves keep loading; `JsonUnitySaveSerializer` remains available as a dependency-free fallback.

```
using Cysharp.Threading.Tasks;
using Vareiko.Foundation.Save;

public sealed class ProfileFacade
{
    private readonly ISaveService _save;

    public ProfileFacade(ISaveService save)
    {
        _save = save;
    }

    public UniTask SaveProfileAsync(PlayerProfile profile)
    {
        return _save.SaveAsync("main", "profile", profile);
    }

    public UniTask<PlayerProfile> LoadProfileAsync()
    {
        return _save.LoadAsync("main", "profile", new PlayerProfile());
    }
}
```

Settings

```
using Vareiko.Foundation.Settings;

public sealed class AudioSettingsFacade
{
    private readonly ISettingsService _settings;

    public AudioSettingsFacade(ISettingsService settings)
    {
        _settings = settings;
    }

    public void SetMusicVolume(float volume01)
    {
        var next = _settings.Current;
        next.MusicVolume = volume01;
        _settings.Apply(next, saveImmediately: true);
    }
}
```

Scene Flow

```
using Cysharp.Threading.Tasks;
using UnityEngine.SceneManagement;
using Vareiko.Foundation.SceneFlow;

public sealed class SceneRouter
{
    private readonly ISceneFlowService _sceneFlow;

    public SceneRouter(ISceneFlowService sceneFlow)
    {
        _sceneFlow = sceneFlow;
    }

    public UniTask OpenGameplayAsync()
    {
        return _sceneFlow.LoadSceneAsync("Gameplay", LoadSceneMode.Single, setActive: true);
    }
}
```

Feature Flags

```
using Cysharp.Threading.Tasks;
using Vareiko.Foundation.Features;

public sealed class FeatureGate
{
    private readonly IFeatureFlagService _flags;

    public FeatureGate(IFeatureFlagService flags)
    {
        _flags = flags;
    }

    public async UniTask<bool> IsNewEconomyEnabledAsync()
    {
        await _flags.RefreshAsync();
        return _flags.IsEnabled("economy.v2", false);
    }
}
```

Cloud Commands

```
using System;
using Cysharp.Threading.Tasks;
using Vareiko.Foundation.Backend;

public sealed class RunBackendFacade
{
    private readonly ICloudCommandService _commands;

    public RunBackendFacade(ICloudCommandService commands)
    {
        _commands = commands;
    }

    public UniTask<CloudCommandResponse> StartRunAsync(string payloadJson)
    {
        CloudCommandRequest request = new CloudCommandRequest(
            commandName: "StartRun",
            idempotencyKey: GenerateUuidV7(),
            correlationId: Guid.NewGuid().ToString(),
        );
    }
}
```

```

        requestVersion: "1",
        payloadJson: payloadJson ?? "{}",
        playerId: "player-1",
        clientUnixMs: DateTimeOffset.UtcNow.ToUnixTimeMilliseconds());

    return _commands.ExecuteAsync(request);
}

private static string GenerateUuidV7()
{
    return "01890f2e-5c5b-7b2f-a4ab-2f5bdcf18a22";
}
}

```

Deterministic RNG

```

using Vareiko.Foundation.Rng;

public sealed class RunRngFacade
{
    private readonly IDeterministicRngService _rng;

    public RunRngFacade(IDeterministicRngService rng)
    {
        _rng = rng;
    }

    public void InitializeRun(int rootSeed)
    {
        _rng.Initialize(rootSeed);
    }

    public int RollNodeIndex(int maxExclusive)
    {
        IDeterministicRngStream stream = _rng.CreateStream("run.nodes");
        return stream.NextInt(0, maxExclusive);
    }
}

```

7. Core Primitives (3.0)

Tick service

Central ordered update loop — replaces per-service `Update()` loops and coroutines:

```

using Vareiko.Foundation;
using Vareiko.Foundation.Time;

public sealed class SpawnDirector
{
    private readonly ITickService _tick;
    private readonly CompositeDisposable _subscriptions = new CompositeDisposable();

    public SpawnDirector(ITickService tick)
    {
        _tick = tick;
    }

    public void StartRun()
    {
        _subscriptions.Add(_tick.RegisterTick(OnTick, order: 0));
    }
}

```

```

        _subscriptions.Add(_tick.Repeat(1.5f, SpawnWave));
        _subscriptions.Add(_tick.Delay(60f, FinishRun, useUnscaledTime: true));
    }

    public void StopRun()
    {
        _subscriptions.Clear();
    }

    private void OnTick(float deltaTime) { /* ... */ }
    private void SpawnWave() { /* ... */ }
    private void FinishRun() { /* ... */ }
}

```

Listeners run in explicit order; timers accumulate scaled or unscaled time and catch up after long frames; `IsPaused` gates everything. In `EditMode` tests drive `TickService.Advance(deltaTime, unscaledDeltaTime)` manually.

Object pooling

```
using Vareiko.Foundation.Pooling;
```

```

ComponentPool<Projectile> pool = new ComponentPool<Projectile>(projectilePrefab, poolRoot, maxSize: 64,
Projectile projectile = pool.Get();           // activated instance
pool.Release(projectile);                     // deactivated, reparented, reused

```

`ObjectPool<T>` pools plain classes (factory + get/release/destroy callbacks, double-release detection); `GetScoped(out item)` returns a using-scope that releases automatically.

Result

```

public Result<PlayerProfile> ParseProfile(string raw)
{
    return _serializer.TryDeserialize(raw, out PlayerProfile profile)
        ? Result<PlayerProfile>.Success(profile)
        : Result<PlayerProfile>.Fail("Profile payload is corrupted.");
}

```

`Result/Result<T>` (namespace `Vareiko.Foundation`) is the default for new APIs. Domain results that carry error codes or retry flags (backend, cloud sync) keep their own types.

Custom app states and generic FSM

`AppState` is a string-backed struct: the well-known states (`Boot`, `MainMenu`, `Gameplay`, `Error`, ...) are unchanged, and hosts mint their own:

```

private static readonly AppState MetaShop = new AppState("MetaShop");

_appStateMachine.TryEnter(MetaShop);           // flows through the standard lifecycle rules

```

For any other state set use `StateMachine<TState>` directly (transition guard + change event):

```

StateMachine<RunPhase> fsm = new StateMachine<RunPhase>(RunPhase.Warmup, (from, to) => to != RunPhase.W
fsm.StateChanged += (from, to) => Debug.Log($"{from} -> {to}");
fsm.TryEnter(RunPhase.Combat);

```

8. Custom Bootstrap Tasks

Add game-specific boot logic by implementing `IBootstrapTask`.

```
using System.Threading;
using Cysharp.Threading.Tasks;
using Vareiko.Foundation.Bootstrap;
using Vareiko.Foundation.Save;

public sealed class LoadProfileBootstrapTask : IBootstrapTask
{
    private readonly ISaveService _save;

    public int Order => 100;
    public string Name => "Load Player Profile";

    public LoadProfileBootstrapTask(ISaveService save)
    {
        _save = save;
    }

    public async UniTask ExecuteAsync(Cancellation_token cancellation_token)
    {
        _ = await _save.LoadAsync("main", "profile", new PlayerProfile(), cancellation_token);
    }
}
```

Bind it in your project installer or a domain installer.

9. Monetization and Comms Stack

IAP

Main contract: `IInAppPurchaseService`

```
1 InitializeAsync()
1 GetCatalogAsync()
1 PurchaseAsync(productId)
1 RestorePurchasesAsync()
```

Ads

Main contract: `IAdsService`

```
1 InitializeAsync()
1 LoadAsync(placementId)
1 ShowAsync(placementId)
```

Use `IMonetizationPolicyService` to gate ad/purchase actions before calling providers.

Push

Main contract: `IPushNotificationService`

```
1 InitializeAsync()
```



```
1 RequestPermissionAsync()  
1 GetDeviceTokenAsync()  
1 SubscribeAsync(topic)  
1 UnsubscribeAsync(topic)
```

10. External Bridge Integrations

External Ads Bridge

For non-Unity mediation SDKs:

```
using Cysharp.Threading.Tasks;  
using Vareiko.Foundation.Ads;  
  
public static class AdsSdkBridgeInstaller  
{  
    public static void Install()  
    {  
        ExternalAdsBridge.SetInitializeHandler(_ => UniTask.FromResult(AdsInitializeResult.Succeed()));  
        ExternalAdsBridge.SetLoadHandler((placementId, _) => UniTask.FromResult(AdLoadResult.Succeed(pl  
        ExternalAdsBridge.SetShowHandler((placementId, _) => UniTask.FromResult(AdShowResult.Succeed(pl  
    }  
}
```

External Attribution Bridge

For AppsFlyer/Adjust/other attribution SDKs:

```
using System.Collections.Generic;  
using Cysharp.Threading.Tasks;  
using Vareiko.Foundation.Attribution;  
  
public static class AttributionSdkBridgeInstaller  
{  
    public static void Install()  
    {  
        ExternalAttributionBridge.SetTrackEventHandler((eventName, properties, _) =>  
        {  
            IReadOnlyDictionary<string, string> payload = properties;  
            // Forward payload to your attribution SDK here.  
            return UniTask.FromResult(AttributionTrackResult.Succeed(eventName));  
        });  
  
        ExternalAttributionBridge.SetTrackRevenueHandler((data, _) =>  
        {  
            // Forward revenue to your attribution SDK here.  
            return UniTask.FromResult(AttributionRevenueTrackResult.Succeed(data.ProductId, data.Curren  
        });  
    }  
}
```

At shutdown/domain reload call:

```
1 ExternalAdsBridge.ClearHandlers()  
1 ExternalAttributionBridge.ClearHandlers()
```

11. Observability and Diagnostics

Use:

- 1 `IDiagnosticsService` for live snapshot access;
- 1 `IDiagnosticsSnapshotExportService.ExportAsync(label)` for QA/support exports.

Monitor key signals in `SignalBus` for:

- 1 startup validation;
- 1 monetization decisions/failures;
- 1 push permission/topic flow;
- 1 attribution and ads bridge failures.

12. Testing Strategy

Recommended levels:

- 1 Runtime unit tests per module (service contracts + provider mapping).
- 2 Integration smoke tests for bootstrap + scene flow + save/settings.
- 3 PlayMode smoke for scene wiring and audio/UI critical path.

Use package test assembly `Tests/Runtime` as the default pattern for new modules.

13. Release Checklist

- 1 Run `Tools/Vareiko/Foundation/Validate Project`.
- 2 Ensure package version and changelog are aligned.
- 3 Fix all UI registry errors before release:
- 4 duplicate non-empty UI ids
- 5 empty `UIScreen / UIWindow` ids
- 6 active scene template items in `UIItemCollectionBinder`
- 7 non-empty ids on item templates
- 8 Review UI validator warnings:
- 9 active `LayoutGroup / ContentSizeFitter` on runtime collection containers
- 10 decorative `Graphic.RaycastTarget` without a parent `Selectable`
- 11 Ensure no compile errors on clean import.
- 12 Ensure required runtime/editor tests pass.
- 13 Tag release (`VX.Y.Z`) after commit.

14. Common Pitfalls

- 1 Missing provider define/sdks with production provider selected.
- 1 Enabling consent-required modules without loaded consent state.

- 1 Registering bridge handlers too late (after provider init).
- 1 Skipping FoundationSceneInstaller in gameplay scenes.

If you follow the starter flow and keep provider-heavy modules in fallback mode first, integration is usually stable from day one.